

Compression d'image par DCT et stockage CSR en C++

Conception du code et représentation des données

Romain BEN, Karim ZRIG | MAM4 Polytech Nice Sophia, projet C++ | septembre 2026

Code source : <https://github.com/BnRomain/jpeg-compression>, dossier `cpp/submission/`

1. Objectif

En MAM3, nous avons programmé en Python une compression d'image inspirée de JPEG : DCT sur des blocs 8×8 , quantification, suppression des hautes fréquences et stockage des coefficients au format creux CSR avec `scipy`, le tout présenté dans une application Streamlit. Ce projet réécrit l'ensemble en C++20, sans bibliothèque de calcul, sous la forme d'un programme en ligne de commande, `jpeg_csr`. Le but de la semaine étant d'apprendre le C++, ce rapport ne revient que brièvement sur l'algorithme. Il explique surtout comment nous avons représenté chaque objet du problème en C++ (image, bloc, matrice de quantification, matrice creuse, fichier compressé) et quelles notions du cours ont guidé ces choix. Le code source, rendu avec ce rapport, est commenté section par section : nous n'en citons ici que de courts extraits.

2. L'algorithme en bref

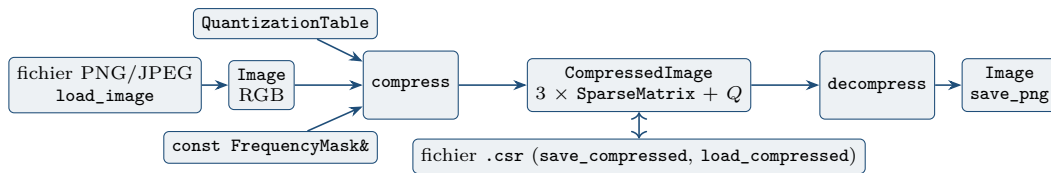
Pour chaque canal R, G, B et chaque bloc M d'intensités centrées dans $[-128, 127]$:

$$D = P M P^T, \quad P_{k,i} = \frac{C_k}{2} \cos \frac{(2i+1)k\pi}{16}, \quad \hat{D}_{k,l} = \text{trunc}\left(\frac{D_{k,l}}{\alpha Q_{k,l}}\right),$$

avec $C_0 = 1/\sqrt{2}$ et $C_k = 1$ sinon. $\hat{D}_{k,l}$ est ensuite annulé si $|\hat{D}_{k,l}| < S$ (seuil) ou si le masque le rejette : le masque carré de la version Python garde $k, l < F$, le masque triangulaire du sujet $k + l < F$. Les coefficients d'un canal forment une matrice presque vide, stockée en CSR. La décompression calcule $\tilde{M} = P^T(\hat{D} \odot \alpha Q)P + 128$, borné à $[0, 255]$; comme P est orthogonale, aucune inversion de matrice n'est nécessaire.

3. Organisation du code

Le code rendu tient en trois fichiers : `jpeg.hpp` déclare tous les types et toutes les fonctions en documentant leurs invariants et les exceptions qu'ils peuvent lancer, `jpeg.cpp` en donne les définitions dans le même ordre, et `main.cpp` contient le programme. Les douze sections, de `Matrix8` à la ligne de commande, portent les mêmes numéros dans les deux premiers fichiers. C'est la compilation séparée de la séance 2 : les deux `.cpp` forment deux unités de traduction, compilées avec les options du cours (`-std=c++20 -Wall -Wextra -pedantic`, plus `-O2`) puis liées. Tout est rangé dans l'espace de noms `jpeg` ; les fonctions utilitaires d'une section sont placées dans un espace de noms anonyme, invisible depuis l'autre fichier. La bibliothèque `stb`, qui lit et écrit les fichiers PNG et JPEG, n'est incluse que par `jpeg.cpp` : le reste du programme ne manipule que notre classe `Image`. Le dépôt garde en plus le même code éclaté en onze modules, avec un `Makefile` qui construit `jpeg_csr` et `run_tests` à partir des mêmes objets ; les deux versions produisent des fichiers identiques octet par octet. Le schéma suit le trajet des données ; chaque boîte est un type ou une fonction du code.



4. Représenter les données en C++

4.1 L'image : un seul tableau contigu

```
class Image {
public:
    static constexpr std::size_t channels{3};

    Image(std::size_t width, std::size_t height,
          double value = 0.0);

    double& operator()(std::size_t row,
                       std::size_t col, std::size_t channel);
    const double& operator()(std::size_t row,
                             std::size_t col, std::size_t channel) const;
    // at(): same access, checked
    Image cropped_to_blocks() const;

private:
    std::size_t width_;
    std::size_t height_;
    std::vector<double> pixels_;
};
```

En Python, l'image était un tableau `numpy` de forme $(n_y, n_x, 3)$. Un vecteur de vecteurs imbriqués aurait demandé une allocation par ligne, sans rien empêcher deux lignes d'avoir des tailles différentes. Nous rangeons donc les $w \times h \times 3$ intensités dans un seul `std::vector<double>`, ligne par ligne : la valeur `(row, col, channel)` est à l'indice $(row \cdot w + col) \cdot 3 + channel$. La mémoire est contiguë, parcourue dans l'ordre des boucles, et la cohérence entre dimensions et données se réduit à un invariant : `pixels_.size() = w \times h \times 3`, avec des dimensions non nulles. Le constructeur l'établit, les données sont privées et aucune méthode ne change la taille. Le type `double` reprend le `float64` de `numpy` et évite des conversions dans la DCT.

L'accès passe par `operator()`, surchargé comme l'`operator[]` de `IntArray` (TD 4) et de `State` (TD 5) : la version non `const` renvoie `double&` pour écrire, la version `const` renvoie `const double&` pour lire une image constante, par

exemple dans `compress(const Image&, ...)`. Comme `std::vector`, la classe offre aussi `at()`, qui vérifie les bornes et lance `std::out_of_range`; les boucles de calcul utilisent `operator()`, sans vérification. Le rognage aux multiples de 8 est une méthode `const` qui renvoie une nouvelle image : l'originale reste intacte.

4.2 Le bloc 8×8 et la DCT : une taille fixe, aucune allocation

Un bloc de pixels, la matrice de passage P et un bloc de coefficients sont tous des matrices 8×8 de réels : un seul type, `Matrix8`, les représente. Ses 64 coefficients sont dans un `std::array<double, 64>`, dont la taille est connue à la compilation : contrairement à `std::vector`, aucune allocation dynamique n'a lieu. Ce choix compte, car une image 512×512 contient $3 \times 64 \times 64 = 12288$ blocs et chaque bloc crée plusieurs matrices temporaires. Le produit est un opérateur libre `operator*(const Matrix8&, const Matrix8&)`, qui n'a besoin que de l'interface publique et permet d'écrire `p_ * block * p_transposed_` comme la formule mathématique.

La classe `Dct` calcule P et P^T une seule fois, dans son constructeur, puis les réutilise pour chaque bloc à travers deux méthodes `const`, `forward` et `inverse`. Nous y avons rencontré une règle du C++ : les membres sont initialisés dans l'ordre de leur déclaration dans la classe, et non dans celui de la liste d'initialisation. `p_` doit donc être déclaré avant `p_transposed_`, qui est calculé à partir de lui ; un commentaire le signale dans l'en-tête.

4.3 La matrice de quantification : un invariant qui fixe un type

`QuantizationTable` encapsule un `Matrix8` de diviseurs, avec pour invariant que tous valent au moins 1. Le constructeur, déclaré `explicit` pour interdire la conversion implicite d'un `Matrix8` en table, le vérifie et lance `std::invalid_argument` sinon ; aucune méthode ne modifie ensuite la table. La condition est écrite `!(d >= 1)` plutôt que `d < 1`, pour rejeter aussi une valeur NaN. Cet invariant a une conséquence directe sur la représentation : un coefficient de DCT vérifie $|D_{k,l}| \leq 8 \times 128 = 1024$, donc, divisé par un nombre au moins égal à 1, il tient dans un `std::int16_t`, le type stocké dans la matrice creuse (l'équivalent de `.astype(np.int16)`). Les quatre matrices étudiées en MAM3 sont des fonctions membres statiques (`standard()`, `uniform()`, `low_frequencies()`, `high_frequencies()`) qui nomment chaque manière de construire une table. Le facteur de qualité est une méthode `const`, `scaled(alpha)`, qui renvoie une nouvelle table : son constructeur revérifie l'invariant, car $\alpha < 1$ peut produire un diviseur inférieur à 1.

4.4 La matrice creuse CSR, écrite à la main

```
class SparseMatrix {
public:
    // from the dense matrix, only read
    SparseMatrix(std::size_t rows, std::size_t cols,
        const std::vector<std::int16_t>& dense);

    // from the arrays read in a .csr file:
    // taken by value, then moved
    SparseMatrix(std::size_t rows, std::size_t cols,
        std::vector<std::int16_t> values,
        std::vector<std::int32_t> column_indices,
        std::vector<std::int32_t> row_pointers);
    ...
private:
    std::size_t rows_;
    std::size_t cols_;
    std::vector<std::int16_t> values_;
    std::vector<std::int32_t> column_indices_;
    std::vector<std::int32_t> row_pointers_;
};
```

`scipy.sparse.csr_matrix` est remplacée par la classe `SparseMatrix`, qui reprend exactement le format CSR : `values_` contient les `nnz` valeurs non nulles, ligne par ligne, `column_indices_` la colonne de chacune, et la ligne i occupe les positions `[row_pointers_[i], row_pointers_[i + 1])` de ces deux tableaux. Les types sont ceux de `scipy` (valeurs sur 16 bits, indices sur 32 bits), si bien que les tailles mémoire des deux versions sont directement comparables. Chaque canal est une matrice creuse de la taille de l'image : le coefficient (k, l) du bloc de coin (*top, left*) est rangé en $(top + k, left + l)$, comme dans la version Python.

L'invariant du format est précis : `row_pointers_` a `rows + 1` éléments, commence à 0, croît et se termine à `nnz` ; dans une ligne, les colonnes sont strictement croissantes et dans `[0, cols)` ; aucune valeur stockée n'est nulle.

Les deux constructeurs correspondent aux deux usages. Le premier construit le CSR à partir de la matrice dense des coefficients quantifiés, qu'il ne fait que lire : il la reçoit par `const&`. Le second reçoit les trois tableaux relus dans un fichier. Il les prend par valeur puis les déplace dans l'objet avec `std::move` (séance 4) ; comme l'appelant lui transmet ses propres vecteurs avec `std::move`, aucun des éléments n'est recopié. Ce constructeur appelle `check_invariant()`, qui rejette des tableaux incohérents.

4.5 L'image compressée et le fichier .csr

`CompressedImage` est une composition (TD 5) : les dimensions, la table de quantification, indispensable à la décompression, et un `std::vector` de trois `SparseMatrix`. Son constructeur vérifie que les dimensions sont des multiples de 8 et que les trois canaux ont la taille de l'image. Le fichier `.csr`, équivalent du `.npz` de l'application, contient une signature, les dimensions, les 64 diviseurs puis, pour chaque canal, `nnz` et les trois tableaux. L'écriture binaire repose sur des fonctions `write` et `read` surchargées pour `std::uint32_t`, `double` et les vecteurs d'entiers : le compilateur choisit la version d'après le type de l'argument (séance 2), et `reinterpret_cast` présente chaque valeur comme une suite d'octets. À la relecture, les dimensions et `nnz` sont bornés avant toute allocation, pour qu'un fichier corrompu ne réserve pas plusieurs gigaoctets ; les constructeurs de `SparseMatrix`, `QuantizationTable` et `CompressedImage` revérifient ensuite leurs invariants. Un fichier incohérent est ainsi rejeté par une exception au lieu de produire une image faussée.

4.6 Les réglages de la ligne de commande

Les réglages lus par `parse_options` forment un simple agrégat, `struct Options`, dont chaque champ a une valeur par défaut, celle de l'application Python. Nous l'avons laissé en `struct` volontairement : il ne porte pas d'invariant propre, la validité des valeurs étant contrôlée par les classes qui les utilisent. Les choix discrets (commande, matrice, masque) sont des `enum class` : contrairement à un entier ou à une chaîne, une valeur hors liste ne compile pas, et `-Wall` signale un `switch` qui oublierait un cas.

5. Conception : les notions du cours

5.1 Invariants, exceptions et signatures

Chaque classe suit le schéma du TD 3 : données `private`, constructeur qui établit l'invariant, souvent grâce à une fonction de vérification appelée dans la liste d'initialisation (comme `checked_capacity` pour `IntArray`), et exception sinon. Un objet qui existe est donc toujours valide, et les fonctions de calcul n'ont rien à révéifier. Toutes les erreurs (arguments, fichiers, invariants) remontent jusqu'à un unique bloc `try/catch` dans `main`, qui affiche le message et renvoie 1. Les signatures expriment l'intention (séance 1) : `compress` lit ses arguments par `const&` sans les copier, `add_salt_and_pepper(Image&, ...)` modifie l'image de l'appelant, les petites valeurs (`int`, `double`) passent par copie et les accesseurs sont `const` et `noexcept`.

5.2 Ressources : RAII et règle de zéro

```
// jpeg.cpp section 9, anonymous namespace
class StbPixels {
public:
    explicit StbPixels(const std::string& path);
    ~StbPixels(); // stbi_image_free(data_)

    StbPixels(const StbPixels&) = delete;
    StbPixels& operator=(const StbPixels&) = delete;
    ...
private:
    int width_{};
    int height_{};
    unsigned char* data_{nullptr};
};
```

Toutes nos classes composent des types qui gèrent déjà leur mémoire (`std::vector`, `std::array`) : aucune n'écrit de destructeur ni d'opération de copie ou de déplacement, et celles que génère le compilateur sont correctes. C'est la règle de zéro (TD 5 et TD de synthèse) : le programme ne contient aucun `new` ni `delete`. La seule ressource manuelle est le tableau de pixels alloué par `stbi_load`, qui doit être libéré par `stbi_image_free`.

Nous l'avons confié à la classe `StbPixels`, sur le modèle de `IntArray` (TD 3) et de la session Gmsh du TD de synthèse : le constructeur acquiert le tableau, le destructeur le libère, y compris si une exception interrompt `load_image`. La copie est interdite avec `= delete`, car deux objets libéreraient sinon le même tableau, le problème observé au TD 4. De la même façon, `save_compressed` n'appelle jamais `close()` : le destructeur de `std::ofstream` ferme le fichier, même en cas d'erreur.

5.3 Polymorphisme : un masque choisi à l'exécution

```
class FrequencyMask {
public:
    virtual bool keeps(std::size_t k,
                      std::size_t l) const = 0;
    virtual std::string name() const = 0;
    virtual ~FrequencyMask() = default;
};

// main.cpp: the mask lives on the stack
if (options.mask == MaskKind::triangle) {
    const TriangleMask mask(options.cutoff);
    return compress_with_mask(options, mask);
}
const SquareMask mask(options.cutoff);
return compress_with_mask(options, mask);
```

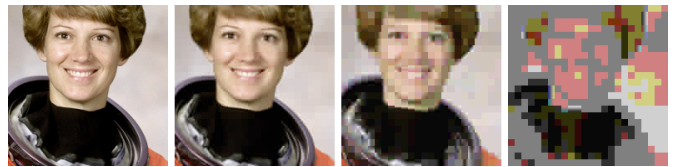
La version Python tronquait les hautes fréquences en carré, le sujet de MAM3 en triangle. Plutôt qu'un paramètre testé dans la boucle de compression, nous avons repris la démarche du TD 6 (`ScalarFunction` et `midpoint`) : l'interface abstraite `FrequencyMask` déclare `keeps(k, l)` virtuelle pure, `SquareMask` et `TriangleMask` la redéfinissent avec `override`, et le destructeur est virtuel. `compress` reçoit un `const FrequencyMask&` et ne nomme aucune stratégie : ajouter une forme de masque ne modifie pas son code.

Dans `main`, le masque demandé est un objet local, construit sur la pile et passé par référence de base : la référence utilise l'objet sans le posséder ni le copier, ce qui évite aussi le slicing étudié au TD 5. Le prix est un appel virtuel par coefficient, faible devant les deux produits matriciels de chaque bloc.

6. Validation et résultats

`make test` lance 17 fonctions de test à base d'`assert` : les tests `pytest` de la version Python (orthogonalité de P , rognage, suppression des hautes fréquences, bornes, conversion CSR), les invariants de chaque classe (un constructeur doit lancer une exception sur une entrée invalide), l'aller-retour par fichier `.csr` et l'analyse de la ligne de commande. L'intégration continue GitHub les relance sous `AddressSanitizer` et `UBSan`, comme en TD, et mesure la couverture : 91 % des lignes du code sont exécutées par les tests et la démonstration. Le script `compare_with_python.py` compresse la même image avec les deux versions : 786 407 coefficients sur 786 432 sont identiques, avec autant de non nuls. Les 25 autres diffèrent de 1 à cause d'un arrondi flottant côté Python, où l'image passe par $/255$ puis $\times 255$: D/Q y vaut par exemple 23,999 999 999 996 au lieu de 24, et la troncature donne 23.

Réglage	Conserv.	Erreur L^2	CSR	Gain
$\alpha = 1$ (Python)	5,79 %	5,43 %	273 Ko	22,5
$\alpha = 5$	2,15 %	11,29 %	105 Ko	58,5
$\alpha = 20$	0,64 %	32,95 %	35 Ko	174,0
Q uniforme (100)	1,65 %	13,85 %	82 Ko	75,1
triangle $F = 2$	2,91 %	11,41 %	140 Ko	43,8



originale, puis $\alpha = 1, 5$ et 20 (zoom)

Image `astronaut.png` (512×512 , 6 144 Ko en `float64`), seuil $S = 2$, masque carré $F = 6$ sauf mention contraire ; le gain est le rapport entre la taille dense et la taille CSR. Comme en MAM3, α règle le compromis entre taux et qualité. Avec 5 % de bruit poivre et sel, l'erreur par rapport à l'image pure passe de 24,0 % à 11,3 % après compression avec un masque triangulaire $F = 4$: la troncature agit comme un filtre passe-bas. Compression et décompression prennent 50 ms en C++ contre 543 ms en Python sur la même machine, soit un facteur 11 : les boucles sont compilées en `-O2` et `Matrix8` n'alloue rien, là où `numpy` est appelé bloc par bloc.

7. Difficultés rencontrées

- **Faire confiance à une bibliothèque C.** L'analyse CodeQL de GitHub a signalé dans `stb` des multiplications d'`int` qui calculent des tailles de tampons et peuvent déborder. Nous ne compilons que les décodeurs PNG, JPEG et BMP, ces calculs ont été refaits en `std::size_t` et les images sont limitées à 16 384 pixels de côté. Cette limite existe à deux endroits, la macro de `stb` et la constante `Image::max_dimension` : un `static_assert` vérifie à la compilation qu'elles restent égales.
- **Les entiers ont une taille.** Les indices CSR sont des `int32`, comme dans `scipy` : le constructeur de `SparseMatrix` vérifie que la matrice reste indexable sur 32 bits avant toute conversion. Les passages entre `int`, `std::size_t` et `double` sont écrits avec `static_cast`, pour que chaque changement de type se voie à la lecture.
- **Garder un niveau d'avertissement élevé.** Compilés avec `-Wall -Wextra -pedantic`, les en-têtes de `stb` produisaient des avertissements qui noyaient les nôtres : les inclure avec `-isystem` les fait taire sans baisser notre niveau d'exigence.
- **Un exécutable portable sous Windows.** Un programme compilé avec MinGW dépend de bibliothèques partagées que d'autres logiciels (Git, MATLAB) installent dans d'autres versions : l'édition de liens statique (`-static`) le rend indépendant du `PATH`.
- **Vérifier un portage.** Le script de comparaison confronte les 786 432 coefficients un à un plutôt que les images reconstruites, ce qui a permis d'isoler les 25 écarts et de remonter à leur cause, un arrondi flottant côté Python.

8. Bilan : ce que le projet nous a appris

Réécrire un programme que nous connaissions nous a permis de nous concentrer sur le C++ lui-même. Trois idées ressortent.

- **Le type porte l'information.** En Python, rien n'empêchait une image de dimensions quelconques ou un coefficient hors de l'`int16` ; en C++, ces hypothèses sont devenues des invariants vérifiés une fois, à la construction, et des types choisis en conséquence.
- **Chaque copie se voit.** Choisir entre `T`, `T&` et `const T&`, entre `std::array` et `std::vector`, ou déplacer plutôt que copier, oblige à savoir ce que coûte chaque ligne, ce que `numpy` nous cachait.
- **La durée de vie structure le code.** RAII et règle de zéro suppriment la gestion mémoire manuelle : la seule ressource brute est enfermée dans une classe de quelques lignes.

Limites et pistes. Le fichier `.csr` n'est pas un vrai JPEG : il manque le parcours zigzag et le codage entropique (RLE, Huffman), et l'espace YCbCr permettrait de sous-échantillonner la chrominance. Côté C++, les pointeurs intelligents, annoncés pour la suite du cours, permettraient de conserver plusieurs masques polymorphes dans un même conteneur.